ASSIGNMENT-9.1

Name : L.Anvitha reddy

Roll No : 2303A51451

Batch : 23

Problem 1:

Consider the following Python function:

def find_max(numbers):

return max(numbers)

Task:

• Write documentation for the function in all three formats:

(a) Docstring

(b) Inline comments

(c) Google-style documentation

• Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style.

• Recommend which documentation style is most effective for a mathematical utilities library and justify your answer.

Code:

```
#docstring
def find_max(numbers):
    """
    Find and return the maximum number in a list of numbers.
    Parameters:
    numbers (list): A list of numbers (integers or floats).
    Returns:
```

```
    int or float: The maximum number in the list.
    """
    return max(numbers)
print(find_max.__doc__)
# Inline comments
def find_max(numbers):
    # Use the built-in max function to find the maximum number in the list
    return max(numbers) # This line returns the maximum number found in the list of numbers provided as input.
print(find_max.__doc__)
#Google style documentation
def find_max(numbers):
    """
    Find and return the maximum number in a list of numbers.
Args:
        numbers (list): A list of numbers (integers or floats).
    Returns:
        int or float: The maximum number in the list.
    Raises:
ValueError: If the input list is empty.
TypeError: If the input is not a list of numbers.
    """
    if not isinstance(numbers, list):
        raise TypeError("Input must be a list of numbers.")
    if not numbers:
        raise ValueError("Input list cannot be empty.")
    return max(numbers)
print(find_max.__doc__)
```

Problem 2: Consider the following Python function:

```
def login(user, password, credentials):
return credentials.get(user) == password
```

Task:

1. Write documentation in all three formats.

2. Critically compare the approaches.

3. Recommend which style would be most helpful for new developers onboarding a project, and justify your choice.

Code:

```
#docstring
def login(user, password,credentials):
    """Logs in a user if the provided credentials are correct.
Args:
        user (str): The username of the user trying to log in.
        password (str): The password of the user trying to log in.
        credentials (dict): A dictionary containing valid usernames and their corresponding passwords.
    Returns:
        bool: True if the login is successful, False otherwise.
        """
    return credentials.get(user) == password
print(login.__doc__)
# Inline comments
def login(user, password,credentials):
    # Check if the provided username exists in the credentials dictionary and if the corresponding password matches the provided password
    return credentials.get(user) == password # This line checks if the username exists in the credentials and if the password matches, returning True for a successful login and False otherwise.
print(login.__doc__)
#Google style documentation
def login(user, password,credentials):
    """
    Logs in a user if the provided credentials are correct.
```

```
Args:
        user (str): The username of the user trying to log in.
        password (str): The password of the user trying to log in.
        credentials (dict): A dictionary containing valid usernames and their corresponding passwords.
    Returns:
        bool: True if the login is successful, False otherwise.
    Raises:
TypeError: If user or password is not a string, or if credentials is not a dictionary.
    """
    if not isinstance(user, str) or not isinstance(password, str):
        raise TypeError("User and password must be strings.")
    if not isinstance(credentials, dict):
        raise TypeError("Credentials must be a dictionary.")
    return credentials.get(user) == password # This line checks if the username exists in the credentials and if the password matches, returning True for a successful login and False otherwise.
print(login.__doc__)
```

Problem 3: Calculator (Automatic Documentation Generation)

Task: Design a Python module named calculator.py and

demonstrate automatic documentation generation.

Instructions:

1. Create a Python module calculator.py that includes the

following functions, each written with appropriate docstrings:

o add(a, b) – returns the sum of two numbers

o subtract(a, b) – returns the difference of two numbers

o multiply(a, b) – returns the product of two numbers

o divide(a, b) – returns the quotient of two numbers

2. Display the module documentation in the terminal using

Python’s documentation tools.

3. Generate and export the module documentation in HTML

format using the pydoc utility, and open the generated HTML

file in a web browser to verify the output.

Code:

```
"""
calculator.py
A simple calculator module demonstrating automatic documentation generation.
It provides basic arithmetic operations.
"""
def add(a, b):
    """
    Returns the sum of two numbers.
    Parameters:
    a (int or float): First number
    b (int or float): Second number
    Returns:
    int or float: Sum of a and b
    """
    return a + b

def subtract(a, b):
    """
    Returns the difference of two numbers.
    Parameters:
    a (int or float): First number
    b (int or float): Second number
    Returns:
    int or float: Difference of a and b
    """
    return a - b

def multiply(a, b):
```

```
    """
    Returns the product of two numbers.
    Parameters:
    a (int or float): First number
    b (int or float): Second number
    Returns:
    int or float: Product of a and b
    """
    return a * b

def divide(a, b):
    """
    Returns the quotient of two numbers.
    Parameters:
    a (int or float): Numerator
    b (int or float): Denominator
    Returns:
    int or float: Quotient of a and b
    Raises:
ValueError: If b is zero
    """
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b
```

Problem 4: Conversion Utilities Module

Task:

1. Write a module named conversion.py with functions:

o decimal_to_binary(n)

o binary_to_decimal(b)

o decimal_to_hexadecimal(n)

2. Use Copilot for auto-generating docstrings.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format and open it in a

browser.

Code:

```
"""
conversion.py
A utility module providing number system conversion functions.Includes decimal to binary, binary to decimal, and decimal to hexadecimal conversions.
"""


def decimal_to_binary(n):
    """
    Convert a decimal integer to its binary representation.
    Parameters:
    n (int): Decimal number
    Returns:
    str: Binary representation of n
    """
    return bin(n)[2:]


def binary_to_decimal(b):
    """
    Convert a binary string to its decimal representation.
    Parameters:
    b (str): Binary number as string
    Returns:
    int: Decimal equivalent of b
    """
```

```
    return int(b, 2)


def decimal_to_hexadecimal(n):
    """
    Convert a decimal integer to its hexadecimal representation.
    Parameters:
    n (int): Decimal number
    Returns:
    str: Hexadecimal representation of n
    """
    return hex(n)[2:].upper()
```

Problem 5 – Course Management Module

Task:

1. Create a module course.py with functions:

o add_course(course_id, name, credits)

o remove_course(course_id)

o get_course(course_id)

2. Add docstrings with Copilot.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format and open it in a browser.

Code:

```
"""
course.py
A simple course management module that allows adding, removing,
and retrieving course details.
```

```
"""
# Dictionary to store course data
courses = {}
def add_course(course_id, name, credits):
    """
    Add a new course to the course collection.
    Parameters:
course_id (str): Unique course identifier
    name (str): Course name
    credits (int): Number of credits for the course
    Returns:
    str: Confirmation message
    """
    courses[course_id] = {"name": name, "credits": credits}
    return f"Course {course_id} added successfully."

def remove_course(course_id):
    """
    Remove a course from the course collection.
    Parameters:
course_id (str): Unique course identifier
    Returns:
    str: Confirmation or error message
    """
    if course_id in courses:
        del courses[course_id]
        return f"Course {course_id} removed successfully."
    return "Course not found."

def get_course(course_id):
    """
```

```
    Retrieve course details by course ID.

    Parameters:

course_id (str): Unique course identifier

    Returns:

dict or str: Course details or error message

    """

    return courses.get(course_id, "Course not found.")
```